

Statistical and Computational Models

Lecture 5: Penalization

Daniel de Kadt

Department of Government, Cornell University · Fall 2026

Penalization and overfitting

Ridge regression

The lasso

The elastic net

Penalization

Minimising training MSE is not a good route to learning – over-fitting.

Penalization

Minimising training MSE is not a good route to learning – over-fitting.

This becomes particularly problematic as the class of possible functions grows:

- more flexibility means more chance to over-fit. . .
- stretches our data at an increasing rate. . .
- increasingly problematic for: high p , small n

Penalization

Minimising training MSE is not a good route to learning – over-fitting.

This becomes particularly problematic as the class of possible functions grows:

- more flexibility means more chance to over-fit...
- stretches our data at an increasing rate...
- increasingly problematic for: high p , small n

Penalization restricts the function class (aka regularization, shrinkage).

Penalization

Minimising training MSE is not a good route to learning – over-fitting.

This becomes particularly problematic as the class of possible functions grows:

- more flexibility means more chance to over-fit...
- stretches our data at an increasing rate...
- increasingly problematic for: high p , small n

Penalization restricts the function class (aka regularization, shrinkage).

Very helpful for:

- variable selection in cases with many options
- making future data collection less arbitrary
- covariate selection for improving efficiency in causal designs
- finding the right covariates to pair-match on before randomization

Adding more covariates

So far we have worked with 200 age-by-state observations. Suppose we now merge in some administrative data.

Adding more covariates

So far we have worked with 200 age-by-state observations. Suppose we now merge in some administrative data.

This gives us 150 new columns – median income, urbanicity, broadband share, 2020 turnout, and demographics.

```
Z <- merged_file           # 150 new columns
bz <- c(9.0, -6.6, 4.2, rep(0, 147)) # three of them matter
y <- f0(age) + gam * (mail - 0.5) + Z %*% bz + rnorm(n, 0, sig)
```

Adding more covariates

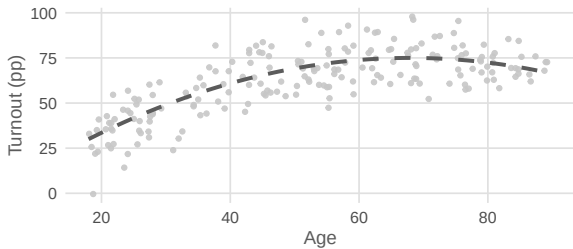
So far we have worked with 200 age-by-state observations. Suppose we now merge in some administrative data.

This gives us 150 new columns – median income, urbanicity, broadband share, 2020 turnout, and demographics.

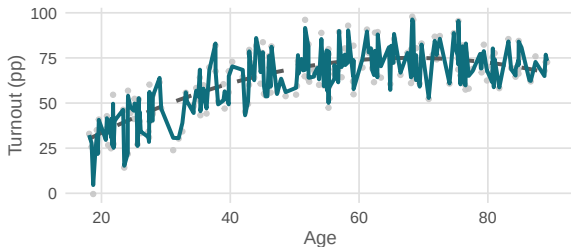
```
Z <- merged_file           # 150 new columns
bz <- c(9.0, -6.6, 4.2, rep(0, 147)) # three of them matter
y <- f0(age) + gam * (mail - 0.5) + Z %*% bz + rnorm(n, 0, sig)
```

Only three of the 150 coefficients actually matter for our outcome variable. But how would we learn that?

Trying OLS

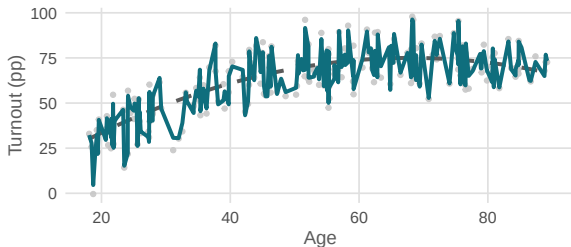


Trying OLS



Suppose we fit OLS on all 200 observations with all 153 columns as features, and project out $\hat{y}$.

Trying OLS



Suppose we fit OLS on all 200 observations with all 153 columns as features, and project out $\hat{y}$.

This looks like overfitting – indeed, we get a training MSE of 12.4, but a test MSE 176.0 (against a noise floor of 49).

Choosing a subset

One way to reduce the complexity of this model is to choose only a **subset of features**.

Choosing a subset

One way to reduce the complexity of this model is to choose only a **subset of features**.

Best subset: fit every model possible – this would be 2^{153} models.

Choosing a subset

One way to reduce the complexity of this model is to choose only a **subset of features**.

Best subset: fit every model possible – this would be 2^{153} models.

Forward stepwise: 11,782 fits, and greedy.

Choosing a subset

One way to reduce the complexity of this model is to choose only a **subset of features**.

Best subset: fit every model possible – this would be 2^{153} models.

Forward stepwise: 11,782 fits, and greedy.

In either approach, we fit a lot of models and score each by cross-validation, keeping only the best.

Choosing a subset

One way to reduce the complexity of this model is to choose only a **subset of features**.

Best subset: fit every model possible – this would be 2^{153} models.

Forward stepwise: 11,782 fits, and greedy.

In either approach, we fit a lot of models and score each by cross-validation, keeping only the best.

Note: AIC and BIC do the same job by adjusting the training MSE instead; we will get there soon.

Choosing a subset

One way to reduce the complexity of this model is to choose only a **subset of features**.

Best subset: fit every model possible – this would be 2^{153} models.

Forward stepwise: 11,782 fits, and greedy.

In either approach, we fit a lot of models and score each by cross-validation, keeping only the best.

Note: AIC and BIC do the same job by adjusting the training MSE instead; we will get there soon.

With large n and p and more complex models these approaches become computationally unreasonable.

Roadmap

Penalization and overfitting

Ridge regression

The lasso

The elastic net

A **norm** measures the size of a vector. We already know one:

$$\|\mathbf{b}\|_2 = \sqrt{b_1^2 + b_2^2 + \dots + b_p^2} = \sqrt{\mathbf{b}^\top \mathbf{b}}$$

Read: the **Euclidean** length of $\mathbf{b}$ – Pythagoras, in p dimensions. For example, $(3, 4)^\top$ has length $\sqrt{9 + 16} = 5$.

Norms

A **norm** measures the size of a vector. We already know one:

$$\|\mathbf{b}\|_2 = \sqrt{b_1^2 + b_2^2 + \dots + b_p^2} = \sqrt{\mathbf{b}^\top \mathbf{b}}$$

Read: the **Euclidean** length of $\mathbf{b}$ – Pythagoras, in p dimensions. For example, $(3, 4)^\top$ has length $\sqrt{9 + 16} = 5$.

Two things follow. $\|\mathbf{b}\|_2^2 = \mathbf{b}^\top \mathbf{b}$, the ridge penalty as the derivation writes it. And a difference has a length: $\|\mathbf{x}_i - \mathbf{x}_0\|_2$ is how k -NN finds neighbours in more than one dimension.

Norms

A **norm** measures the size of a vector. We already know one:

$$\|\mathbf{b}\|_2 = \sqrt{b_1^2 + b_2^2 + \dots + b_p^2} = \sqrt{\mathbf{b}^\top \mathbf{b}}$$

Read: the **Euclidean** length of $\mathbf{b}$ – Pythagoras, in p dimensions. For example, $(3, 4)^\top$ has length $\sqrt{9 + 16} = 5$.

Two things follow. $\|\mathbf{b}\|_2^2 = \mathbf{b}^\top \mathbf{b}$, the ridge penalty as the derivation writes it. And a difference has a length: $\|\mathbf{x}_i - \mathbf{x}_0\|_2$ is how k -NN finds neighbours in more than one dimension.

Question: what does a small $\|\mathbf{b}\|_2$ say about $\mathbf{b}$?

Norms

A **norm** measures the size of a vector. We already know one:

$$\|\mathbf{b}\|_2 = \sqrt{b_1^2 + b_2^2 + \dots + b_p^2} = \sqrt{\mathbf{b}^\top \mathbf{b}}$$

Read: the **Euclidean** length of $\mathbf{b}$ – Pythagoras, in p dimensions. For example, $(3, 4)^\top$ has length $\sqrt{9 + 16} = 5$.

Two things follow. $\|\mathbf{b}\|_2^2 = \mathbf{b}^\top \mathbf{b}$, the ridge penalty as the derivation writes it. And a difference has a length: $\|\mathbf{x}_i - \mathbf{x}_0\|_2$ is how k -NN finds neighbours in more than one dimension.

Question: what does a small $\|\mathbf{b}\|_2$ say about $\mathbf{b}$? Every b_j is small.

Two more ways to size a vector:

$$\|\mathbf{b}\|_1 = \sum_{j=1}^p |b_j|, \quad \|\mathbf{b}\|_0 = \sum_{j=1}^p I(b_j \neq 0)$$

Read: ℓ_1 adds up absolute sizes, no squaring. ℓ_0 counts the non-zeros and ignores size altogether (not a true norm).

Two more ways to size a vector:

$$\|\mathbf{b}\|_1 = \sum_{j=1}^p |b_j|, \quad \|\mathbf{b}\|_0 = \sum_{j=1}^p I(b_j \neq 0)$$

Read: ℓ_1 adds up absolute sizes, no squaring. ℓ_0 counts the non-zeros and ignores size altogether (not a true norm).

For our example $\mathbf{b} = (3, -4)^\top$:

Two more ways to size a vector:

$$\|\mathbf{b}\|_1 = \sum_{j=1}^p |b_j|, \quad \|\mathbf{b}\|_0 = \sum_{j=1}^p I(b_j \neq 0)$$

Read: ℓ_1 adds up absolute sizes, no squaring. ℓ_0 counts the non-zeros and ignores size altogether (not a true norm).

For our example $\mathbf{b} = (3, -4)^\top$:

- $\|\mathbf{b}\|_2 = 5$
- $\|\mathbf{b}\|_1 = 7$
- $\|\mathbf{b}\|_0 = 2$.

Penalised least squares

Recall our motivation: penalising complicated functions avoids overfitting. The penalty is one number, made of two pieces:

$$\underbrace{\lambda}_{\text{how much size counts}} \times \underbrace{\|\mathbf{b}\|_2^2}_{\text{how big } \mathbf{b} \text{ is: } \sum_j b_j^2}$$

Read: big coefficients add to the objective; λ sets how much. The intercept b_0 is not in $\mathbf{b}$.

Penalised least squares

Recall our motivation: penalising complicated functions avoids overfitting. The penalty is one number, made of two pieces:

$$\underbrace{\lambda}_{\text{how much size counts}} \times \underbrace{\|\mathbf{b}\|_2^2}_{\text{how big } \mathbf{b} \text{ is: } \sum_j b_j^2}$$

Read: big coefficients add to the objective; λ sets how much. The intercept b_0 is not in $\mathbf{b}$.

Add it to the in-sample loss, and minimise the sum:

$$\hat{\beta}_{\lambda}^R = \arg \min_{\mathbf{b}} \left\{ \underbrace{\sum_{i=1}^n \left(y_i - b_0 - \sum_{j=1}^p x_{ij} b_j \right)^2}_{\text{RSS}(\mathbf{b}): \text{fit the data}} + \underbrace{\lambda \|\mathbf{b}\|_2^2}_{\text{stay small}} \right\}$$

Penalised least squares

Recall our motivation: penalising complicated functions avoids overfitting. The penalty is one number, made of two pieces:

$$\underbrace{\lambda}_{\text{how much size counts}} \times \underbrace{\|\mathbf{b}\|_2^2}_{\text{how big } \mathbf{b} \text{ is: } \sum_j b_j^2}$$

Read: big coefficients add to the objective; λ sets how much. The intercept b_0 is not in $\mathbf{b}$.

Add it to the in-sample loss, and minimise the sum:

$$\hat{\beta}_{\lambda}^R = \arg \min_{\mathbf{b}} \left\{ \underbrace{\sum_{i=1}^n \left(y_i - b_0 - \sum_{j=1}^p x_{ij} b_j \right)^2}_{\text{RSS}(\mathbf{b}): \text{fit the data}} + \underbrace{\lambda \|\mathbf{b}\|_2^2}_{\text{stay small}} \right\}$$

The larger λ , the more aggressive the **shrinkage** toward zero. At $\lambda = 0$, what do we get?

Penalised least squares

Recall our motivation: penalising complicated functions avoids overfitting. The penalty is one number, made of two pieces:

$$\underbrace{\lambda}_{\text{how much size counts}} \times \underbrace{\|\mathbf{b}\|_2^2}_{\text{how big } \mathbf{b} \text{ is: } \sum_j b_j^2}$$

Read: big coefficients add to the objective; λ sets how much. The intercept b_0 is not in $\mathbf{b}$.

Add it to the in-sample loss, and minimise the sum:

$$\hat{\beta}_{\lambda}^R = \arg \min_{\mathbf{b}} \left\{ \underbrace{\sum_{i=1}^n \left(y_i - b_0 - \sum_{j=1}^p x_{ij} b_j \right)^2}_{\text{RSS}(\mathbf{b}): \text{fit the data}} + \underbrace{\lambda \|\mathbf{b}\|_2^2}_{\text{stay small}} \right\}$$

The larger λ , the more aggressive the **shrinkage** toward zero. At $\lambda = 0$, what do we get? OLS.

Penalised least squares

Recall our motivation: penalising complicated functions avoids overfitting. The penalty is one number, made of two pieces:

$$\underbrace{\lambda}_{\text{how much size counts}} \times \underbrace{\|\mathbf{b}\|_2^2}_{\text{how big } \mathbf{b} \text{ is: } \sum_j b_j^2}$$

Read: big coefficients add to the objective; λ sets how much. The intercept b_0 is not in $\mathbf{b}$.

Add it to the in-sample loss, and minimise the sum:

$$\hat{\beta}_{\lambda}^R = \arg \min_{\mathbf{b}} \left\{ \underbrace{\sum_{i=1}^n \left(y_i - b_0 - \sum_{j=1}^p x_{ij} b_j \right)^2}_{\text{RSS}(\mathbf{b}): \text{fit the data}} + \underbrace{\lambda \|\mathbf{b}\|_2^2}_{\text{stay small}} \right\}$$

The larger λ , the more aggressive the **shrinkage** toward zero. At $\lambda = 0$, what do we get? OLS. As $\lambda \rightarrow \infty$, what do we get?

Penalised least squares

Recall our motivation: penalising complicated functions avoids overfitting. The penalty is one number, made of two pieces:

$$\underbrace{\lambda}_{\text{how much size counts}} \times \underbrace{\|\mathbf{b}\|_2^2}_{\text{how big } \mathbf{b} \text{ is: } \sum_j b_j^2}$$

Read: big coefficients add to the objective; λ sets how much. The intercept b_0 is not in $\mathbf{b}$.

Add it to the in-sample loss, and minimise the sum:

$$\hat{\beta}_{\lambda}^R = \arg \min_{\mathbf{b}} \left\{ \underbrace{\sum_{i=1}^n \left(y_i - b_0 - \sum_{j=1}^p x_{ij} b_j \right)^2}_{\text{RSS}(\mathbf{b}): \text{ fit the data}} + \underbrace{\lambda \|\mathbf{b}\|_2^2}_{\text{stay small}} \right\}$$

The larger λ , the more aggressive the **shrinkage** toward zero. At $\lambda = 0$, what do we get? OLS. As $\lambda \rightarrow \infty$, what do we get? The intercept-only model.

Solving for $\hat{\beta}_{\lambda}^R$

This is called **ridge regression**. Let's solve for $\hat{\beta}_{\lambda}^R$ as we did for OLS: find $\hat{\beta}$ to minimise the 'objective function' (RSS).

We add our penalty $\lambda \mathbf{b}^T \mathbf{b}$ to define a new objective function, then proceed:

Solving for $\hat{\beta}_{\lambda}^R$

This is called **ridge regression**. Let's solve for $\hat{\beta}_{\lambda}^R$ as we did for OLS: find $\hat{\beta}$ to minimise the 'objective function' (RSS).

We add our penalty $\lambda \mathbf{b}^T \mathbf{b}$ to define a new objective function, then proceed:

Step 1: expand.

$$\text{RSS}(\mathbf{b}) + \lambda \mathbf{b}^T \mathbf{b} = \mathbf{y}^T \mathbf{y} - 2 \mathbf{b}^T \mathbf{X}^T \mathbf{y} + \mathbf{b}^T (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) \mathbf{b}$$

Solving for $\hat{\beta}_{\lambda}^R$

This is called **ridge regression**. Let's solve for $\hat{\beta}_{\lambda}^R$ as we did for OLS: find $\hat{\beta}$ to minimise the 'objective function' (RSS).

We add our penalty $\lambda \mathbf{b}^T \mathbf{b}$ to define a new objective function, then proceed:

Step 1: expand.

$$\text{RSS}(\mathbf{b}) + \lambda \mathbf{b}^T \mathbf{b} = \mathbf{y}^T \mathbf{y} - 2 \mathbf{b}^T \mathbf{X}^T \mathbf{y} + \mathbf{b}^T (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) \mathbf{b}$$

Step 2: differentiate with respect to $\mathbf{b}$, and set to zero.

$$-2 \mathbf{X}^T \mathbf{y} + 2 (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) \mathbf{b} = \mathbf{0}$$

Solving for $\hat{\beta}_\lambda^R$

This is called **ridge regression**. Let's solve for $\hat{\beta}_\lambda^R$ as we did for OLS: find $\hat{\beta}$ to minimise the 'objective function' (RSS).

We add our penalty $\lambda \mathbf{b}^\top \mathbf{b}$ to define a new objective function, then proceed:

Step 1: expand.

$$\text{RSS}(\mathbf{b}) + \lambda \mathbf{b}^\top \mathbf{b} = \mathbf{y}^\top \mathbf{y} - 2 \mathbf{b}^\top \mathbf{X}^\top \mathbf{y} + \mathbf{b}^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \mathbf{b}$$

Step 2: differentiate with respect to $\mathbf{b}$, and set to zero.

$$-2\mathbf{X}^\top \mathbf{y} + 2(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \mathbf{b} = \mathbf{0}$$

Step 3: solve for $\mathbf{b}$.

$$\hat{\beta}_\lambda^R = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

QED

Solving for $\hat{\beta}_{\lambda}^R$

For OLS we required $\mathbf{X}$ of **full column rank**.

Solving for $\hat{\beta}_{\lambda}^R$

For OLS we required $\mathbf{X}$ of **full column rank**.

With $\lambda > 0$ on the diagonal, the inverse always exists. So unlike OLS, you don't need $n > p$. And no **multicollinearity** problems.

Solving for $\hat{\beta}_\lambda^R$

For OLS we required $\mathbf{X}$ of **full column rank**.

With $\lambda > 0$ on the diagonal, the inverse always exists. So unlike OLS, you don't need $n > p$. And no **multicollinearity** problems.

But $\mathbb{E}[\hat{\beta}_\lambda^R] \neq \beta^\dagger$ for any $\lambda > 0$. Why?

Solving for $\hat{\beta}_\lambda^R$

For OLS we required $\mathbf{X}$ of **full column rank**.

With $\lambda > 0$ on the diagonal, the inverse always exists. So unlike OLS, you don't need $n > p$. And no **multicollinearity** problems.

But $\mathbb{E}[\hat{\beta}_\lambda^R] \neq \beta^\dagger$ for any $\lambda > 0$. Why?

We add a little **bias** to reduce **variance**.

We have so far not spoken about how we choose λ :

λ is another **tuning parameter** – changing its value will essentially change the search space for our learner.

We have so far not spoken about how we choose λ :

λ is another **tuning parameter** – changing its value will essentially change the search space for our learner.

Ideally, we choose it by some sensible criterion:

We have so far not spoken about how we choose λ :

λ is another **tuning parameter** – changing its value will essentially change the search space for our learner.

Ideally, we choose it by some sensible criterion: **K -fold** or leave-one-out **cross-validation**, minimising the CV MSE.

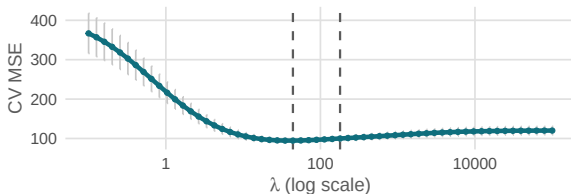
We have so far not spoken about how we choose λ :

λ is another **tuning parameter** – changing its value will essentially change the search space for our learner.

Ideally, we choose it by some sensible criterion: **K -fold** or leave-one-out **cross-validation**, minimising the CV MSE.

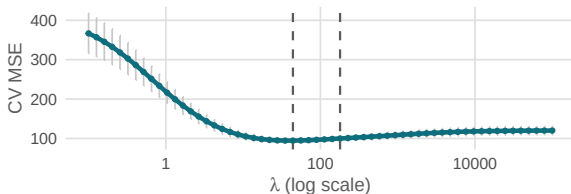
Same goal as choosing **k** , but now tuning something else.

λ by cross-validation



1. A grid of λ , coarse and wide.
2. 10-fold CV at each one, every step **inside** the fold.
3. Take the minimiser, then refit on our sample.

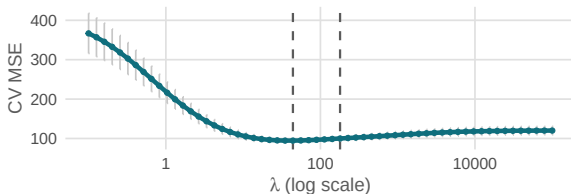
λ by cross-validation



1. A grid of λ , coarse and wide.
2. 10-fold CV at each one, every step **inside** the fold.
3. Take the minimiser, then refit on our sample.

Bars are one fold SE, and the minimum sits at $\lambda = 44$, CV MSE 94.6.

λ by cross-validation

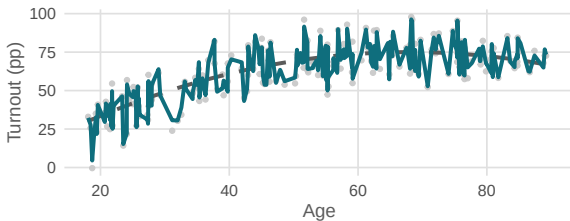


1. A grid of λ , coarse and wide.
2. 10-fold CV at each one, every step **inside** the fold.
3. Take the minimiser, then refit on our sample.

Bars are one fold SE, and the minimum sits at $\lambda = 44$, CV MSE 94.6.

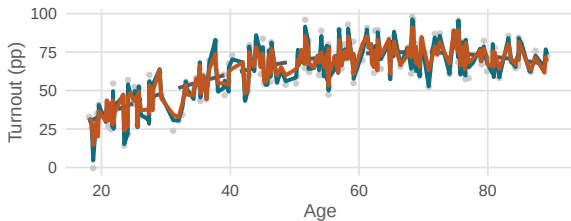
The second dashed rule is called the **one-standard-error rule**: the largest λ within one SE of the minimum. Why not take the simpler model?

What ridge does with them



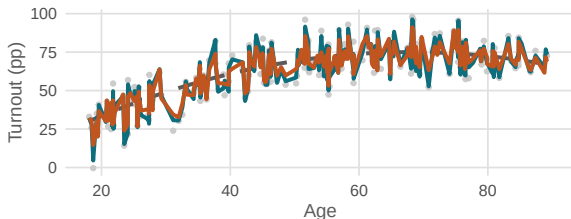
The green curve is OLS.

What ridge does with them



The green curve is OLS. Orange is ridge regression at $\lambda = 44$, chosen by CV: training MSE 29.1, test MSE 82.3.

What ridge does with them



The green curve is OLS. Orange is ridge regression at $\lambda = 44$, chosen by CV: training MSE 29.1, test MSE 82.3.

Ridge is not scale invariant, so we must **standardise** the variables first (this is one part of what is called 'feature engineering', and it is very important in the world of machine learning and statistical learning).

Roadmap

Penalization and overfitting

Ridge regression

The lasso

The elastic net

The lasso

The lasso is very similar to ridge:

$$\hat{\beta}_{\lambda}^L = \arg \min_{\mathbf{b}} \left\{ \sum_{i=1}^n \left(y_i - b_0 - \sum_{j=1}^p x_{ij} b_j \right)^2 + \lambda \sum_{j=1}^p |b_j| \right\}$$

The lasso

The lasso is very similar to ridge:

$$\hat{\beta}_{\lambda}^L = \arg \min_{\mathbf{b}} \left\{ \sum_{i=1}^n \left(y_i - b_0 - \sum_{j=1}^p x_{ij} b_j \right)^2 + \lambda \sum_{j=1}^p |b_j| \right\}$$

The difference is purely in the penalty: $\lambda \|\mathbf{b}\|_1$ instead of $\lambda \|\mathbf{b}\|_2^2$.

The lasso

The lasso is very similar to ridge:

$$\hat{\beta}_{\lambda}^L = \arg \min_{\mathbf{b}} \left\{ \sum_{i=1}^n \left(y_i - b_0 - \sum_{j=1}^p x_{ij} b_j \right)^2 + \lambda \sum_{j=1}^p |b_j| \right\}$$

The difference is purely in the penalty: $\lambda \|\mathbf{b}\|_1$ instead of $\lambda \|\mathbf{b}\|_2^2$.

This change has important consequences. The closed form solution is gone, because the absolute value is **kinked** at zero, and derivatives stop at kinks.

The lasso

The lasso is very similar to ridge:

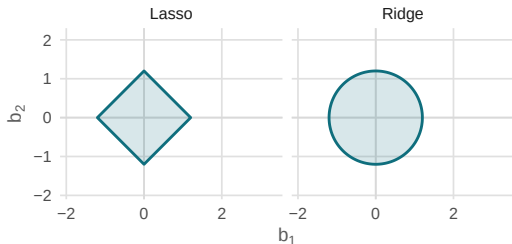
$$\hat{\beta}_{\lambda}^L = \arg \min_{\mathbf{b}} \left\{ \sum_{i=1}^n \left(y_i - b_0 - \sum_{j=1}^p x_{ij} b_j \right)^2 + \lambda \sum_{j=1}^p |b_j| \right\}$$

The difference is purely in the penalty: $\lambda \|\mathbf{b}\|_1$ instead of $\lambda \|\mathbf{b}\|_2^2$.

This change has important consequences. The closed form solution is gone, because the absolute value is **kinked** at zero, and derivatives stop at kinks.

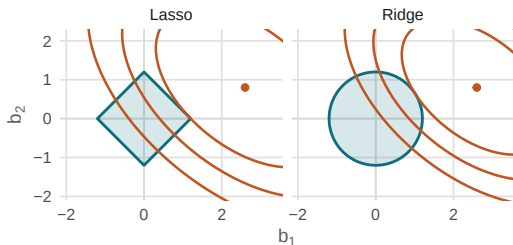
This is the **lasso** (least absolute shrinkage and selection operator).

Why does the lasso produce zeros?



A situation with $p = 2$. The shaded area is the 'allowed parameter space' for a given λ : lasso $\sum_j |b_j| \leq s$, ridge $\sum_j b_j^2 \leq s$.

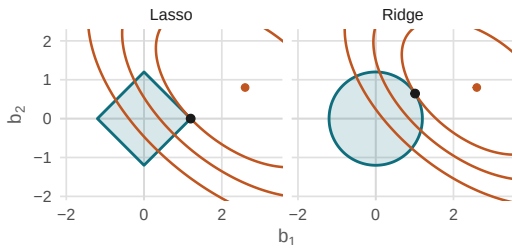
Why does the lasso produce zeros?



A situation with $p = 2$. The shaded area is the 'allowed parameter space' for a given λ : lasso $\sum_j |b_j| \leq s$, ridge $\sum_j b_j^2 \leq s$.

$\hat{\beta}$ is the OLS estimate of (b_1, b_2) . The ellipses are choices of $\mathbf{b}$ with equal RSS. Further from $\hat{\beta} \rightsquigarrow$ larger RSS.

Why does the lasso produce zeros?

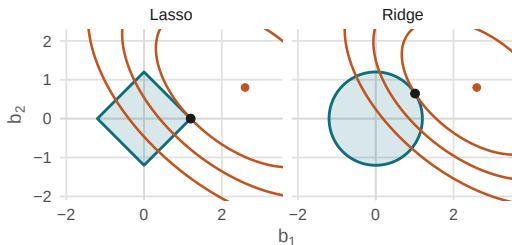


A situation with $p = 2$. The shaded area is the 'allowed parameter space' for a given λ : lasso $\sum_j |b_j| \leq s$, ridge $\sum_j b_j^2 \leq s$.

$\hat{\beta}$ is the OLS estimate of (b_1, b_2) . The ellipses are choices of $\mathbf{b}$ with equal RSS. Further from $\hat{\beta} \rightsquigarrow$ larger RSS.

The ridge and lasso estimates are the first intersection between the shaded area and the ellipses.

Why does the lasso produce zeros?

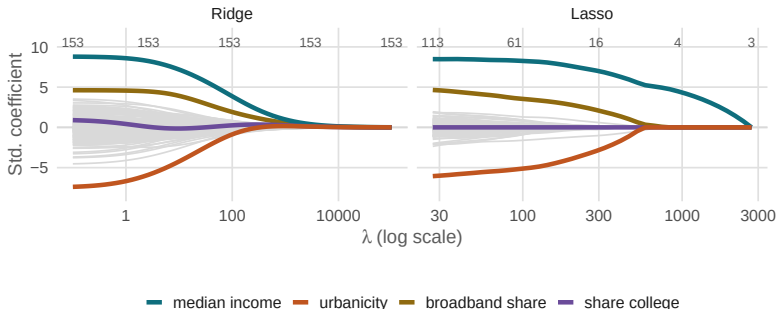


A situation with $p = 2$. The shaded area is the 'allowed parameter space' for a given λ : lasso $\sum_j |b_j| \leq s$, ridge $\sum_j b_j^2 \leq s$.

$\hat{\beta}$ is the OLS estimate of (b_1, b_2) . The ellipses are choices of $\mathbf{b}$ with equal RSS. Further from $\hat{\beta} \rightsquigarrow$ larger RSS.

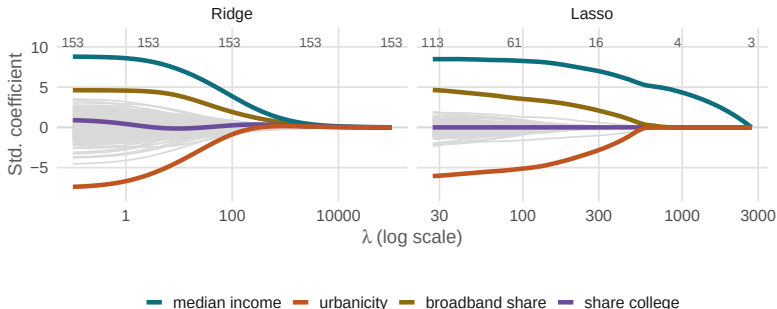
The ridge and lasso estimates are the first intersection between the shaded area and the ellipses. The lasso's sits on a **corner**.

Comparing ridge and lasso paths



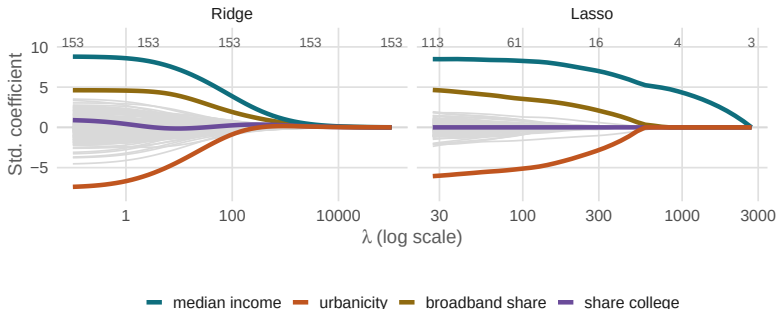
Each **path** is one of the 150 penalised coefficients as λ grows; the counts on top are the non-zero totals.

Comparing ridge and lasso paths



Each **path** is one of the 150 penalised coefficients as λ grows; the counts on top are the non-zero totals. 153 in the ridge panel, but 113 down to just 3 for lasso.

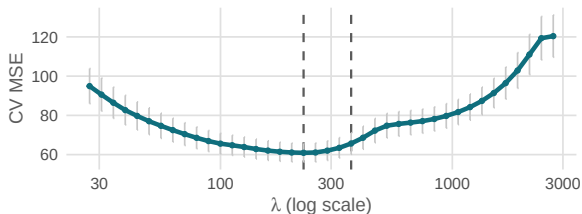
Comparing ridge and lasso paths



Each **path** is one of the 150 penalised coefficients as λ grows; the counts on top are the non-zero totals. 153 in the ridge panel, but 113 down to just 3 for lasso.

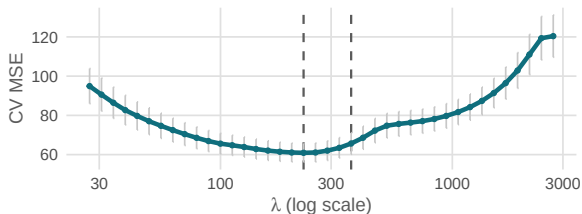
Note: with highly correlated variables the lasso will tend (almost randomly) to choose one and reject the others.

λ by cross-validation



CV for the lasso – the minimum sits at $\lambda = 228$, CV MSE 60.9.

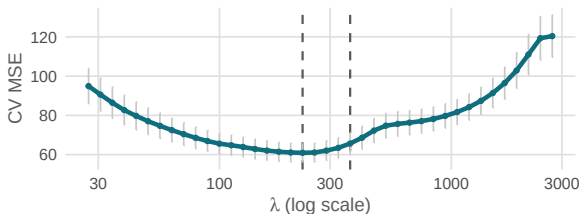
λ by cross-validation



CV for the lasso – the minimum sits at $\lambda = 228$, CV MSE 60.9.

The one-standard-error rule pushes us to use only 12 columns instead of 22 (CV cannot tell these two apart, so we choose the simpler model).

λ by cross-validation



CV for the lasso – the minimum sits at $\lambda = 228$, CV MSE 60.9.

The one-standard-error rule pushes us to use only 12 columns instead of 22 (CV cannot tell these two apart, so we choose the simpler model).

General advice: use `{glmnet}::cv.glmnet(X, y, alpha = 1)`. so ba

Roadmap

Penalization and overfitting

Ridge regression

The lasso

The elastic net

The elastic net

The lasso picks one of a correlated group, almost at random. And with $p > n$ it keeps at most n columns.

The elastic net

The lasso picks one of a correlated group, almost at random. And with $p > n$ it keeps at most n columns.

Zou and Hastie (2005): use **both** penalties.

$$\hat{\beta}_{\lambda, \alpha}^{EN} = \arg \min_{\mathbf{b}} \left\{ \text{RSS}(\mathbf{b}) + \lambda \left[\alpha \sum_{j=1}^p |b_j| + (1 - \alpha) \sum_{j=1}^p b_j^2 \right] \right\}$$

Read: $\alpha = 1$ is the lasso, $\alpha = 0$ is ridge. The ℓ_1 part still sends coefficients to zero. The ℓ_2 part makes the penalty strictly convex, so identical columns get **identical** coefficients.

The elastic net

The lasso picks one of a correlated group, almost at random. And with $p > n$ it keeps at most n columns.

Zou and Hastie (2005): use **both** penalties.

$$\hat{\beta}_{\lambda, \alpha}^{EN} = \arg \min_{\mathbf{b}} \left\{ \text{RSS}(\mathbf{b}) + \lambda \left[\alpha \sum_{j=1}^p |b_j| + (1 - \alpha) \sum_{j=1}^p b_j^2 \right] \right\}$$

Read: $\alpha = 1$ is the lasso, $\alpha = 0$ is ridge. The ℓ_1 part still sends coefficients to zero. The ℓ_2 part makes the penalty strictly convex, so identical columns get **identical** coefficients.

Check: duplicate **median income** and refit. The lasso gives 6.27 and 1.14; the elastic net gives 3.28 and 3.28.

The elastic net

The lasso picks one of a correlated group, almost at random. And with $p > n$ it keeps at most n columns.

Zou and Hastie (2005): use **both** penalties.

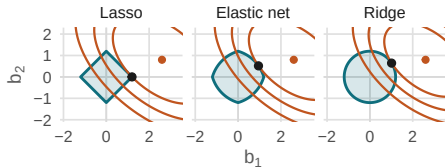
$$\hat{\beta}_{\lambda, \alpha}^{EN} = \arg \min_{\mathbf{b}} \left\{ \text{RSS}(\mathbf{b}) + \lambda \left[\alpha \sum_{j=1}^p |b_j| + (1 - \alpha) \sum_{j=1}^p b_j^2 \right] \right\}$$

Read: $\alpha = 1$ is the lasso, $\alpha = 0$ is ridge. The ℓ_1 part still sends coefficients to zero. The ℓ_2 part makes the penalty strictly convex, so identical columns get **identical** coefficients.

Check: duplicate **median income** and refit. The lasso gives 6.27 and 1.14; the elastic net gives 3.28 and 3.28.

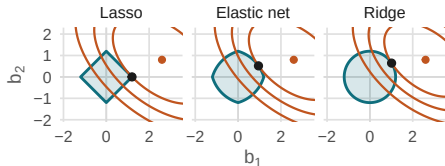
Note: `{glmnet}`'s `alpha` is this α , with a $\frac{1}{2}$ on the ridge part. Two tuning parameters now – a small grid of α , and CV over λ inside each.

The elastic net



Between the diamond and the disc: the **corners** survive, and the sides bulge.

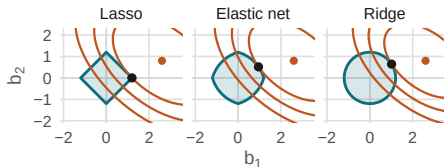
The elastic net



Between the diamond and the disc: the **corners** survive, and the sides bulge.

α	λ	CV MSE	Non-zero	Test MSE
0 (ridge)	44	94.6	153	82.3
0.25	109	92.0	121	76.3
0.5	78	85.2	107	72.8
0.75	105	76.4	72	66.1
1 (lasso)	228	60.9	22	57.2

The elastic net

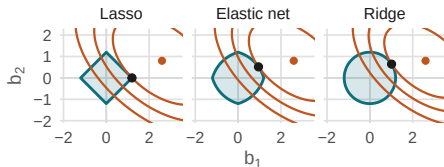


Between the diamond and the disc: the **corners** survive, and the sides bulge.

α	λ	CV MSE	Non-zero	Test MSE
0 (ridge)	44	94.6	153	82.3
0.25	109	92.0	121	76.3
0.5	78	85.2	107	72.8
0.75	105	76.4	72	66.1
1 (lasso)	228	60.9	22	57.2

λ by CV at each α . Here CV walks α to 1: the truth we built is sparse.

The elastic net



Between the diamond and the disc: the **corners** survive, and the sides bulge.

α	λ	CV MSE	Non-zero	Test MSE
0 (ridge)	44	94.6	153	82.3
0.25	109	92.0	121	76.3
0.5	78	85.2	107	72.8
0.75	105	76.4	72	66.1
1 (lasso)	228	60.9	22	57.2

λ by CV at each α . Here CV walks α to 1: the truth we built is sparse. With a correlated group worth keeping, or $p > n$, reach for the elastic net.

Comparison

	Truth	OLS	Ridge	Lasso	Elastic net
median income	9.35	8.81	5.20	7.42	5.50
urbanicity	-6.04	-7.49	-1.95	-3.59	-2.15
broadband share	4.09	4.62	2.63	2.60	2.59
share college	0	0.98	0.08	0	0
λ		0	44	228	78
Non-zero	6	153	153	22	107
$\ \mathbf{b}\ _1$	19.5	203.6	66.0	17.8	44.9
$\ \mathbf{b} - \beta\ _2$		18.83	8.46	3.80	7.22
Training MSE	50.6	12.4	29.1	45.5	33.4
Test MSE	51.1	176.0	82.3	57.2	72.8

Comparison

	Truth	OLS	Ridge	Lasso	Elastic net
median income	9.35	8.81	5.20	7.42	5.50
urbanicity	-6.04	-7.49	-1.95	-3.59	-2.15
broadband share	4.09	4.62	2.63	2.60	2.59
share college	0	0.98	0.08	0	0
λ		0	44	228	78
Non-zero	6	153	153	22	107
$\ \mathbf{b}\ _1$	19.5	203.6	66.0	17.8	44.9
$\ \mathbf{b} - \beta\ _2$		18.83	8.46	3.80	7.22
Training MSE	50.6	12.4	29.1	45.5	33.4
Test MSE	51.1	176.0	82.3	57.2	72.8

Non-zero lasso estimates are biased toward zero, and their standard errors are an active area of research. The lasso performs well here because the truth we built is **sparse**; if we spread signal across all 150 columns ridge would do better.

The penalty for penalization

Throughout we have 'cheated' a little – we exempted `age`, `age2` and `mail` from penalization!

The penalty for penalization

Throughout we have 'cheated' a little – we exempted `age`, `age2` and `mail` from penalization!

What happens if we include them?

	Truth	OLS	Ridge	Lasso	Elastic net
age	50.2	59.1	5.9	41.3	5.8
age2	-39.1	-47.1	3.3	-30.0	3.2
mail	2.0	2.1	1.8	1.8	1.5
Non-zero	6	153	153	85	99
Test MSE	51.1	176.0	152.7	77.1	141.1

The penalty for penalization

Throughout we have 'cheated' a little – we exempted `age`, `age2` and `mail` from penalization!

What happens if we include them?

	Truth	OLS	Ridge	Lasso	Elastic net
age	50.2	59.1	5.9	41.3	5.8
age2	-39.1	-47.1	3.3	-30.0	3.2
mail	2.0	2.1	1.8	1.8	1.5
Non-zero	6	153	153	85	99
Test MSE	51.1	176.0	152.7	77.1	141.1

Ridge regression eliminates the curved relationship with `age`.

The penalty for penalization

Throughout we have 'cheated' a little – we exempted `age`, `age2` and `mail` from penalization!

What happens if we include them?

	Truth	OLS	Ridge	Lasso	Elastic net
age	50.2	59.1	5.9	41.3	5.8
age2	-39.1	-47.1	3.3	-30.0	3.2
mail	2.0	2.1	1.8	1.8	1.5
Non-zero	6	153	153	85	99
Test MSE	51.1	176.0	152.7	77.1	141.1

Ridge regression eliminates the curved relationship with `age`. The lasso recovers it, however.

The penalty for penalization

Throughout we have 'cheated' a little – we exempted `age`, `age2` and `mail` from penalization!

What happens if we include them?

	Truth	OLS	Ridge	Lasso	Elastic net
age	50.2	59.1	5.9	41.3	5.8
age2	-39.1	-47.1	3.3	-30.0	3.2
mail	2.0	2.1	1.8	1.8	1.5
Non-zero	6	153	153	85	99
Test MSE	51.1	176.0	152.7	77.1	141.1

Ridge regression eliminates the curved relationship with `age`. The lasso recovers it, however.

Remember that the penalty does not know which coefficients are 'true.' It penalizes coefficient size, and size depends on the structure of the column. This is very important!

References

- James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021) *An Introduction to Statistical Learning*, 2nd ed. – **6.1–6.2**. statlearning.com.
- Efron, B. & Hastie, T. (2016) *Computer Age Statistical Inference*, **ch. 7**.
- Hastie, T., Tibshirani, R. & Friedman, J. (2009) *The Elements of Statistical Learning*, 2nd ed. – 3.4.
- Lockhart, R., Taylor, J., Tibshirani, R. J. & Tibshirani, R. (2014) "A Significance Test for the Lasso." *Annals of Statistics* 42(2): 413–468.
- Zou, H. & Hastie, T. (2005) "Regularization and Variable Selection via the Elastic Net." *Journal of the Royal Statistical Society B* 67(2): 301–320.